DAEJEON · LESSON 02

제2차시 · 실무 시나리오 기반 설계 워크플로우 자동화와 Vibe Coding 실습

사람이 목적과 판단 범위를 정하고, AI와 함께 작은 업무 자동화 프로그램을 만듭니다. 파일 정리 자동화를 기본으로 실습하고, 도면 제출 전 빠른 점검 예제로 같은 작업 방식을 확장합니다.

수업시간 수업 150분 **구성** 3개 파트 · 14개 섹션 **실습 프롬프트** 6개 수록 **교보재 기준일** 2026.10.19

제2차시 · 대주제 진행

대주제 2 · 완결

실무 시나리오 기반 맞춤형 제안 이미지 생성 및 워크플로우 자동화

실무 시나리오의 목표와 사람의 판단 범위를 정하고, Vibe Coding으로 설계 워크플로우 자동화를 구현·실행하며 대주제 2를 마무리합니다.

Part 1 · 업무 자동화 이해하고 실습 환경 준비하기

자동화할 일과 사람이 판단할 일을 구분하고 실습 폴더를 연결합니다.

Part 2 · 프로그램의 구조를 만들고 요청 준비하기

자연어 요청에 대한 구조 제안을 읽고 보완할 조건을 정합니다.

Part 3 · Codex로 구현하고 직접 확인하기

구현한 파일을 확인하고 프로그램을 실행해 기대 결과와 실제 결과를 비교합니다.

이번 차시의 학습 목표

1. 사람이 먼저 작업 목표와 범위를 정하고 AI에게 계획을 요청합니다.
2. 자동화할 반복 작업과 사람이 판단할 예외를 구분합니다.
3. 프로그램의 역할과 폴더·파일 구조를 설명하고 AI의 제안을 검토합니다.
4. 허용한 폴더에서 구현을 요청하고 직접 실행한 결과를 기록합니다.

Part 1

업무 자동화 이해하고 실습 환경 준비하기

이번 파트의 결과: 자동화할 일과 사람이 판단할 일을 구분하고 실습 폴더를 연결합니다.

01 지난 차시에서 오늘로 이어가기

지난 차시의 작업 방식을 짧게 돌아보고, 같은 원칙을 오늘의 업무 자동화에 연결합니다.

지난 차시에는 AI가 만든 결과를 바로 사용하는 것이 아니라, 요청과 자료를 확인하고 사람이 조건을 정한 뒤 결과를 다시 검토하는 과정을 익혔습니다.

오늘은 같은 원칙을 프로그램 제작과 업무 자동화에 적용합니다.

지난 차시의 작업 흐름

요청 확인 → 필요한 조건 정리 → 작업 기준 만들기 → AI 요청 작성 → 결과 생성 → 원본과 비교 → 사람이 최종 판단

AI가 도운 일

- 요청과 자료를 빠르게 정리
- 여러 초안과 방향 제안
- 반복적인 작성·정리 지원
- 수정 요청에 따라 결과 보완

사람이 판단한 일

- 작업 목표와 범위 결정
- 확인되지 않은 조건 구분
- 원본과 결과 비교
- 최종 사용·수정·제외 결정

같은 원칙으로 프로그램 만들기

프로그램을 만들 때도 같은 원칙을 사용합니다. AI에게 바로 코드를 만들게 하지 않고, 다음 순서로 필요한 내용을 확인합니다.

무엇을 만들 것인지 → 어디까지 자동화할 것인지 → 어떤 경우 사람이 확인할 것인지 → 어떤 구조로 만들 것인지 → 실제로 제대로 작동하는지

오늘의 작업 흐름

업무 자동화 이해 → Vibe Coding → 실습 환경 준비 → 프로그램 구조 설계 → Codex로 구현 → 실행 → 기대 결과와 실제 결과 비교

지난 차시에는 AI를 활용해 이미지와 작업 기준을 만들고 그 결과를 사람이 확인했습니다.

이번 차시에는 AI와 함께 작은 프로그램을 만들면서, 미리 정한 규칙에 따라 작업을 처리하는 업무 자동화 방식을 알아봅니다.

02

업무 자동화란?

반복되는 일 중 규칙으로 처리할 수 있는 부분을 찾습니다.

> **사람이 반복해서 하는 일정한 작업을 미리 정한 규칙에 따라 프로그램이 대신 처리하도록 만드는 것**

수작업	자동화	사람 판단
파일이 들어올 때마다 사람이 하나씩 확인하고 이동합니다.	정해둔 규칙에 맞는 파일은 프로그램이 자동 처리합니다.	어디에 넣어야 할지 명확하지 않은 파일은 사람이 결정합니다.

건축·디자인 업무에서 찾아보기

- **결과 이미지 분류** — 정해진 파일명 규칙에 맞는 결과 이미지를 모읍니다.
- **지정 폴더 파일 이동** — 선택한 수집함에서 정해진 목적지로 파일을 보냅니다.
- **반복 파일명 처리** — 같은 이름이 들어오면 버전 번호를 붙입니다.
- **보고용 목록 생성** — 정해진 입력 항목으로 파일 목록을 만듭니다.

> **오늘은 작은 파일 정리 자동화 사례 하나를 만듭니다.**
>
> 위 예시를 모두 구현하지 않습니다. 먼저 수집함에 넣은 파일 중 규칙으로 분류할 수 있는 범위를 정합니다.

03 Vibe Coding이란?

원하는 기능을 자연어로 설명하며 AI와 함께 프로그램을 만듭니다.

업무 문제 → 자동화 가능성 → Vibe Coding

파일을 매번 정리하는 문제가 있다면, 어떤 부분을 규칙으로 처리할 수 있는지 먼저 생각합니다. 그다음 원하는 프로그램의 기능과 동작을 자연어로 설명하면서 AI와 함께 설계·구현·수정합니다. 이 방법을 Vibe Coding이라고 부릅니다.

사람이 먼저 정할 여섯 가지

1. 무엇을 만들 것인가?
2. 어떤 기능이 필요한가?
3. 어디까지 자동화할 것인가?
4. 어떤 경우 사람이 판단할 것인가?
5. 어떤 파일에 접근할 수 있는가?
6. 무엇을 확인해야 성공인가?

> **내 업무에 적용하기**
>
> 반복해서 하는 일 하나를 고르고, 자동으로 처리할 부분과 사람이 결정할 부분을 각각 한 문장으로 적습니다. “알아서 만들어 주세요”에 빠진 조건을 찾아봅니다.

04 실습 환경 준비하기

ChatGPT Desktop에서 Codex와 실습 폴더를 연결하고 작업 범위를 확인합니다.

본 실습 · ChatGPT Desktop + Codex

Codex는 사람이 정한 요구사항과 구조를 실제 프로젝트 파일과 코드로 구현하는 도구입니다. 지금은 환경을 준비하고, 구현 전에 계획을 요청하는 방법까지 확인합니다.

1. **ChatGPT Desktop 실행**
   앱을 열고 사용할 계정으로 로그인합니다.
2. **Codex 사용 가능 여부 확인**
   앱의 선택 메뉴에서 Codex 작업을 사용할 수 있는지 확인합니다. 표시와 접근 가능 여부는 앱·계정 환경에 따라 다를 수 있습니다.
3. **실습 프로젝트 폴더 연결**
   이번 실습 전용 로컬 폴더를 선택합니다. 대화에 파일을 첨부하는 것과 프로젝트 폴더를 연결하는 것을 구분합니다.

4. **현재 작업 경로 확인**

   선택한 폴더 경로를 읽고 다른 업무 폴더가 연결되지 않았는지 확인합니다.

5. **읽기·수정 범위 확인**

   앱의 권한 설정을 확인하고, 실습 폴더 밖은 접근하지 않는다고 요청에 적습니다. 요청 문구와 실제 권한 설정을 함께 확인합니다.

6. **변경 전에 계획 요청**

   “아직 파일을 만들거나 수정하지 말고 필요한 기능, 폴더 구조, 변경 예정 파일부터 설명해 주세요.”라고 요청합니다.

> **Node.js LTS · 로컬 프로그램 실행환경**
>
> Node.js는 Codex 자체를 설치하기 위한 필수요건이 아니라 이번 로컬 프로그램을 실행·테스트하기 위한 실행환경입니다. 명령 창에서 `node --version`으로 설치된 버전을 확인합니다.
>
> 제공하는 실행 예제의 기준은 README에 적힌 Node.js 22.23.2 이상인 LTS 버전입니다. 실제 확인한 최소 버전은 22.23.2이며, 모든 버전·PC의 동작을 보증한다는 뜻은 아닙니다.

CLI 방식의 Codex, Git, VS Code, WSL 설치는 이번 기본 실습의 필수조건이 아닙니다.

> **대체 환경 소개 · Antigravity**
>
> 비슷한 에이전트형 개발 작업은 Antigravity와 같은 다른 개발 환경에서도 진행할 수 있습니다. 이번 본 실습은 ChatGPT Desktop + Codex로 진행합니다.

참고 링크

OpenAI 공식 안내

- ChatGPT 앱
- Codex 시작 안내
- 권한 및 작업 범위 안내

실행환경

- Node.js 공식 다운로드

Part 2

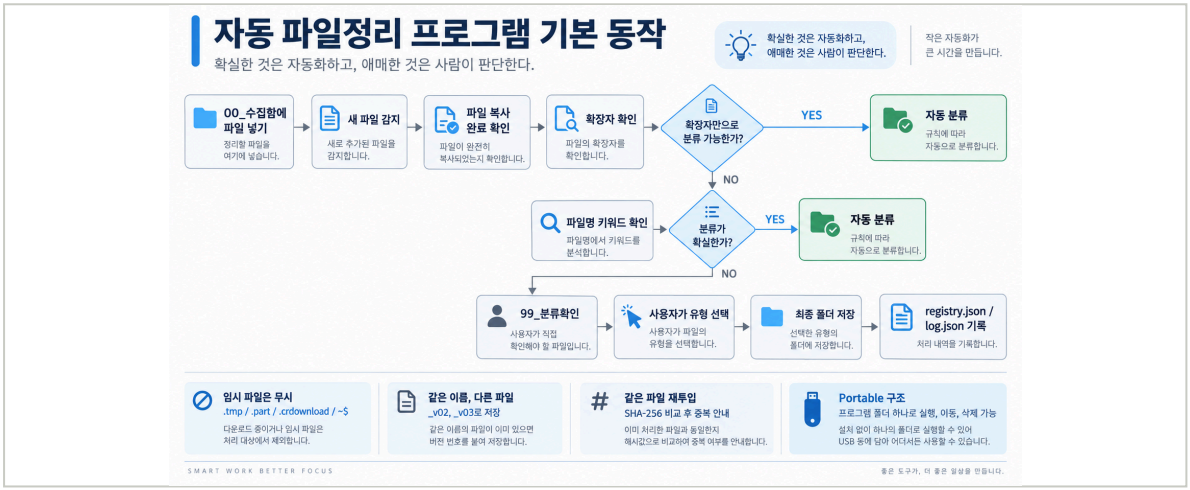
프로그램의 구조를 만들고 요청 준비하기

이번 파트의 결과: 자연어 요청에 대한 구조 제안을 읽고 보완할 조건을 정합니다.

05 프로그램의 큰 구조 이해하기 · Architecture

어떤 기능이 필요하고 서로 어떻게 연결되는지 정합니다.

건축에서 공간의 역할과 연결관계를 먼저 계획하듯, 프로그램도 기능의 역할과 연결관계를 먼저 정합니다. 이것이 프로그램의 큰 구조, Architecture입니다.



확대 보기 원본 PNG 다운로드

파일 자동정리 프로그램의 기본 판단 흐름

처리 전 원본보관, 복사 완료 확인과 분류확인 절차까지 본문 요구조건으로 대조합니다.

01 →
입력 파일
INPUT
사용자가 정리할 파일을 00_수집함에 넣습니다.

02 →
새 파일 확인
WATCHER
새 파일을 살피고 임시 파일이나 복사 중인 파일은 기다립니다.

03 →
분류 규칙 확인
CLASSIFIER
확장자와 파일명이 어떤 규칙에 해당하는지 확인합니다.

04 →
자동처리·사람확인 결정
DECISION
규칙으로 확실하면 자동 처리하고 애매하면 사람에게 돌려줍니다.

05 →
원본 보관·이동·버전 처리
FILE MANAGER
원본을 먼저 보관하고 목적지로 옮기며 같은 이름은 덮어쓰지 않습니다.

06 →
처리 결과 기록
DATA
어떤 파일을 처리했는지와 오류를 기록합니다.

07 →

사용자 확인 화면
UI
결과를 보여주고 사람이 필요한 분류를 선택할 수 있게 합니다.

이 도식은 역할의 연결관계를 보여줍니다. 실제 파일 이동 전에는 복사 완료 확인과 원본보관이 먼저 필요합니다. 영어 이름을 외우기보다 각 역할을 자신의 말로 설명합니다.

> **역할 연결하기**
> 파일명만으로 분류할 수 없으면 어느 역할이 사람에게 판단을 넘기나요? 원본 보관과 결과 기록은 누가 맡나요?

06 기능을 폴더와 파일로 나누기 · Hierarchy

기능의 역할을 실제 폴더와 파일에 나눠 담습니다.

프로그램의 큰 구조 · Architecture

무슨 기능이 필요한가?

폴더와 파일 구성 · Hierarchy

그 기능을 실제 폴더와 파일로 어떻게 나눌 것인가?

```
AI_파일자동정리/
 ├── START.bat / STOP.bat     실행 · 종료
 ├── app/                    프로그램 기능
 |   ├── watcher.js           새 파일 확인
 |   ├── classifier.js        분류 규칙 적용
 |   ├── file-manager.js      원본 · 이동 · 버전 처리
 |   ├── rules.json           사람이 정한 규칙
 |   └── ui/                  사용자 화면
 ├── data/                   처리 상태 · 기록
 ├── 00_수집함/              넣을 파일
 ├── 00_원본보관/            보관한 원본
 ├── 01~11 분류폴더          분류한 결과
 └── 99_분류확인/            사람이 확인할 파일
```

제공하는 실행 예제의 주요 구조입니다. 01~11 분류폴더는 여러 분류 폴더를 줄여 쓴 표기입니다. app에는 기능과 규칙, data에는 처리 상태와 기록을 둡니다.

역할이 놓이는 위치 찾기

“규칙을 바꾸는 곳”, “원본을 보관하는 곳”, “사람이 확인할 파일이 있는 곳”을 찾습니다. 코드 파일명을 암기하는 대신 역할과 위치를 연결합니다.

07

자동화 규칙과 사람 판단 범위 정하기

확실한 것은 자동 처리하고 애매한 것은 사람이 확인하도록 정합니다.



확대 보기 원본 PNG 다운로드

대표 폴더 구조와 분류·예외처리 설명자료

이미지는 대표 예시입니다. 전체 지원 확장자와 판단 규칙은 본문 텍스트 표가 기준입니다. 이미지의 .xyz 예시와 달리 이번 기본 사양은 .xyz를 99_분류확인으로 보냅니다. PDF도 확장자만으로 문서로 확정하지 않습니다.

이 프로그램은 무엇을 보고 분류할까요?

이 프로그램은 AI가 파일 내용을 보고 판단하는 프로그램이 아닙니다. 파일의 확장자와 파일명에 있는 단어를 사람이 미리 정한 규칙과 비교해 분류하는 자동화 예제입니다.

확실한 것은 자동으로 처리하고, 판단 근거가 부족한 것은 사람이 확인합니다.

1

확장자 확인

등록된 직접 분류 확장자라면 정해진 폴더로 자동 분류

.dwg → CAD 도면 · .rvt → BIM · .blend → 3D 모델

→

2

파일명 단서 확인

PNG·JPG·PDF 등은 파일명 단서가 한 분류에만 해당하면 자동 분류

floorplan → 도면 · material / ref → Reference

→

3

불확실하면 사람 확인

단서 없음·서로 다른 분류의 단서 충돌·미등록 확장자는 사람 확인

99_분류확인 → 용도를 확인하고 사람이 분류 선택

PNG, JPG, PDF는 확장자만으로 용도를 알 수 없습니다.

PNG는 도면 이미지·렌더링 결과·재질 참고 이미지·현장사진일 수 있고, PDF는 도면·제안서·보고서·계약서일 수 있습니다. 이 프로그램은 파일 내부를 열어 구분하는 대신 파일명 단서를 추가로 확인합니다.

파일명 단서의 대표 예시

파일명	확인한 단서	규칙에 따른 결과
`office_floorplan.png`	floorplan	도면 후보 → 01_CAD도면 자동 분류
`material_ref_oak.png`	material / ref	Reference 후보 → 03_레퍼런스 자동 분류
`render_final.png`	render / final	이미지 결과 후보 → 04_이미지결과 자동 분류
`project_report.pdf`	report	문서 후보 → 07_문서 자동 분류
`image001.png` `scan003.pdf`	판단 단서 없음	99_분류확인 사람 확인

단서가 충돌하면 사람이 판단합니다.
material_floorplan.png에는 material(Reference 후보)과 floorplan(도면 후보)이 함께 있습니다. 서로 다른 분류가 충돌하므로 임의로 하나를 고르지 않고 99_분류확인으로 보냅니다. 같은 분류에 속한 material / ref가 함께 있는 것은 충돌이 아닙니다.

▼ 실제 분류 규칙 보기

직접 분류 확장자를 먼저 적용하고, 아래 애매한 확장자에만 파일명 단서를 확인합니다. 직접 분류 목록과 애매한 목록에 없는 확장자는 파일명 단서가 있어도 99_분류확인으로 보냅니다.

확장자로 직접 분류하는 유형

CAD / 도면 → 01_CAD도면

`.dwg` `.dxf` `.dgn` `.dwf` `.dwfx`

BIM / 건축 모델 → 02_BIM건축모델

`.rvt` `.rfa` `.rte` `.ifc` `.ifczip` `.pln` `.pla` `.gsm` `.skp`

일반 3D / DCC / 교환 모델 → 05_3D모델

`.blend` `.fbx` `.obj` `.abc` `.glb` `.gltf` `.3ds` `.dae` `.stl` `.ply` `.usd` `.usda` `.usdc` `.usdz` `.3dm` `.c4d` `.max` `.ma` `.mb` `.step` `.stp` `.iges` `.igs` `.sat`

포인트클라우드 / Scan → 06_포인트클라우드

`.e57` `.las` `.laz` `.rcp` `.rcs` `.pts` `.pcd`

문서 → 07_문서

`.doc` `.docx` `.hwp` `.hwpx` `.txt` `.md` `.rtf` `.odt` `.ppt` `.pptx`

영상 → 08_영상

`.mp4` `.mov` `.avi` `.mkv` `.webm` `.m4v`

데이터 → 09_데이터

`.xlsx` `.xls` `.csv` `.json` `.xml` `.yaml` `.yml`

압축 → 10_압축파일

`.zip` `.7z` `.rar` `.tar` `.gz` `.tgz`

디자인 Source → 11_디자인소스

`.psd` `.ai` `.eps` `.indd`

파일명 단서를 추가로 확인하는 확장자

`.png` `.jpg` `.jpeg` `.webp` `.tif` `.tiff` `.bmp` `.gif` `.heic` `.svg` `.pdf`

파일명 키워드 그룹

도면 후보 → 01_CAD도면

`floorplan` `plan` `elevation` `section` `drawing` `detail` `cad` `평면` `평면도` `입면` `입면도` `단면` `단면도` `도면` `상세도`

Reference 후보 → 03_레퍼런스

`ref` `reference` `material` `texture` `lighting` `moodboard` `mood` `재질` `텍스처` `조명` `레퍼런스` `무드보드`

이미지 결과 후보 → 04_이미지결과

`render` `output` `result` `generated` `concept` `selected` `final` `렌더` `결과` `시안` `콘셉트` `선택안`

문서 후보 → 07_문서

`rfp` `brief` `report` `proposal` `contract` `제안서` `보고서` `계약서` `브리프`

영문 키워드는 밑줄(_), 하이픈(-), 공백 등으로 나뉜 단어 전체가 일치해야 합니다. office_plan.png의 plan은 일치하지만 planet.png나 plan01.png는 일치하지 않습니다. 한글 키워드는 파일명에 포함되어 있는지 확인합니다.

임시파일(.tmp / .part / .crdownload), ~$로 시작하는 파일, .gitkeep은 처리 대상에서 제외합니다.

현재 프로그램의 대표 규칙

입력 예	목적지	판단 방식
.dwg	01_CAD도면	확장자 규칙으로 자동 분류
.blend	05_3D모델	확장자 규칙으로 자동 분류
.mp4	08_영상	확장자 규칙으로 자동 분류
PNG / JPG / PDF	파일명 규칙 또는 99_분류확인	알려진 단어가 없거나 여러 분류가 겹치면 사람이 확인
.xyz	99_분류확인	현재 기본 자동분류 규칙에 포함되지 않아 사람이 확인

자동화가 반드시 AI일 필요는 없습니다. 규칙으로 정확하게 처리할 수 있는 반복 작업은 일반 프로그램으로 자동화하고, 규칙만으로 판단하기 어려운 부분은 사람에게 남길 수 있습니다.

Codex / AI · 만드는 과정

- 프로그램 구조 제안
- 코드 작성과 수정
- 테스트 지원

완성된 프로그램 · 실행 과정

- AI 모델을 호출하지 않음
- 외부 AI API를 사용하지 않음
- 사람이 정한 규칙을 반복 실행

자동화의 경계 정하기

이름이 모호한 파일까지 자동으로 보내면 어떤 문제가 생길까요? “사람확인으로 보내기”를 성공 조건에 포함해 봅니다.

08 두 가지 실습 예제와 자연어 첫 요청

두 예제의 목적과 사람 판단 범위를 비교하고 구조부터 제안받습니다.

실습 A를 공통 기본 예제로 진행합니다. 실습 B는 A를 마친 뒤 추가로 체험하거나, 자신의 업무와 더 가까운 예제로 선택할 수 있습니다.

선택한 예제의 구조를 먼저 제안받습니다. 직접 구현을 시도할 수 있고, 준비된 완성 예제를 실행해 기능을 확인할 수도 있습니다.

실습 A · 공통 기본 예제

파일 정리 자동화

여러 형식의 파일을 정해둔 규칙으로 분류하고, 애매한 파일만 사람이 확인합니다.

확장자·파일명 단서를 사람이 정한 규칙에 대조합니다. 실행 중 AI 모델이나 외부 AI API를 호출하지 않습니다.
무엇을 보고 분류하는지 보기

수집함에 직접 넣거나 다른 폴더의 최상위 파일을 가져올 수 있습니다. 두 입력 방법 보기

줄이고 싶은 반복 작업

들어온 파일마다 분류 폴더를 찾고 옮기는 일을 줄입니다.

프로그램이 처리

확장자·파일명 규칙 확인, 원본 보관, 분류·중복 안내·처리 기록

사람이 판단

애매한 파일의 용도, 목적지, 실행 결과와 오류 확인

Windows + Node.js. 프로그램 ZIP을 풀고 START.bat으로 실행합니다.



실습 A 실제 실행 화면 · 이미지를 누르면 크게 볼 수 있습니다.

▼ 실습 A 자연어 첫 요청 보기

실행 플랫폼	OpenAI Codex / ChatGPT 또는 AI 코드 에디터
준비·실행	자연어 기획 요청 → 구조 검토 → 구현 승인 순서로 AI 코딩 도구에 입력합니다.
설계·유추 원리	파일 정리의 불편을 입력·분류·예외·원본 보존 규칙으로 나누고, 구현 전에 질문과 구조안을 받도록 했습니다.
AI로 작성 요청	내 파일 정리 업무를 자동화할 기획 요청문을 짜 줘. 입력 폴더·복사 원칙·애매한 파일의 사람 확인·중복 처리부터 정리하고 지금은 코드를 만들지 마.

PROMPT

파일 정리 자동화 · 구조부터 제안받기

바탕화면에 폴더 하나로 사용할 수 있는
파일 자동정리 프로그램을 만들고 싶어.

00_수집함에 여러 파일을 넣으면
파일 종류에 맞는 폴더로 자동으로 정리됐으면 좋겠어.

다만 PNG, JPG, PDF처럼 용도가 애매한 파일은
마음대로 분류하지 말고
사용자가 확인해서 선택할 수 있게 해줘.

원본은 보관하고,
같은 이름의 파일은 덮어쓰지 않았으면 좋겠고,
프로그램 폴더 전체를 옮기거나 삭제하기 쉬운 형태였으면 해.

아직 만들지는 말고,
먼저 어떤 구조로 만들면 좋을지 설명해줘.
필요한 기능, 폴더 구조, 예외 처리,
테스트해야 할 항목도 같이 제안해줘.

[파일 입력 방법 보완]
기존처럼 00_수집함에 직접 넣는 방법을 유지합니다.
추가로 사용자가 명시적으로 선택하거나 입력한 자료 폴더의 최상위 일반 파일을 읽기 전용으로 확인하고,
목록·용량을 보여준 뒤 “파일 가져오기”를 눌렀을 때만 수집함으로 복사합니다.
외부 원본은 삭제·이동·이름 변경·덮어쓰기하지 않습니다. 하위 폴더·링크·임시파일은 가져오지 않습니다.
복사된 파일은 기존 WATCHER → CLASSIFIER → 사람판단 → 원본보관·버전·SHA 중복 흐름을 재사용합니다.
새 분류기나 새 중복 정책을 만들지 않습니다. 외부 폴더는 읽기만, 쓰기는 프로그램 폴더 안에서만 허용합니다.
Windows의 실제 폴더 선택창과 경로 입력을 제공하고, 한 파일 오류가 다른 파일 가져오기를 중단하지 않게 합니다.
사용자 홈·드라이브·AppData 전체 검색, Registry 변경, 외부 업로드를 하지 않습니다.
직접 입력/가져오기, 원본 이름·수·SHA 불변, 중복, 같은 이름 다른 내용, 오류·빈 폴더를 확인합니다.
아직 구현하지 말고 이 입력 방식까지 포함한 구조부터 설명해 주세요.

실습 B · 추가·선택 예제

도면 제출 전 빠른 점검

여러 도면 파일의 도면번호·도면명·리비전과 확인할 사항을 한 화면에서 살펴봅니다.

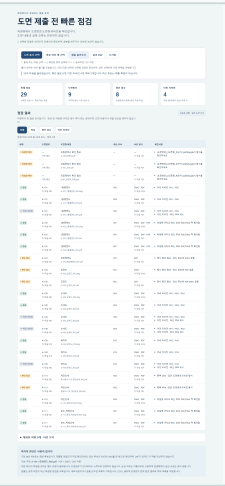
무엇을 보고 점검하는지 보기 · 파일명 기준 최신 후보이며 실제 제출 여부는 사람이 확인합니다.

줄이고 싶은 반복 작업
도면 파일을 하나씩 보며 번호·리비전·형식을 목록에 적는 일을 줄입니다.

프로그램이 처리
파일명 분리, 리비전 숫자 비교, PDF·DWG 짝과 중복 후보 표시

사람이 판단
표시된 후보가 실제 제출 대상인지, 도면 내용과 제출 조건 확인

Chrome / Edge 브라우저에서 실행합니다. 별도 Node.js 설치가 필요하지 않습니다.



실습 B 실제 실행 화면 · 이미지를 누르면 크게 볼 수 있습니다.

▼ 실습 B 자연어 첫 요청 보기

실행 플랫폼	OpenAI Codex / ChatGPT 또는 AI 코드 에디터
준비·실행	자연어 기획 요청 → 구조 검토 → 구현 승인 순서로 AI 코딩 도구에 입력합니다.
설계·유추 원리	도면 확인 업무를 자료 입력·읽을 정보·확인 불가·사람 판단으로 쪼개, 확인 도구의 기능 범위부터 정하도록 했습니다.
AI로 작성 요청	내 도면 확인 업무를 작은 로컬 도구로 만드는 기획 프롬프트를 작성해 줘. 받을 파일·보여줄 정보·자동 판단하지 않을 항목과 필요한 질문부터 정리해 줘.

PROMPT

도면 제출 전 빠른 점검 · 구조부터 제안받기

설계 도면을 제출하기 전에 파일명과 리비전을 한꺼번에 확인할 수 있는 간단한 로컬 프로그램을 만들고 싶어.

도면 폴더를 선택하면 PDF, DWG, DXF 파일명을 읽고 도면번호, 도면명, 리비전과 파일형식을 한 화면의 표로 보여줘.

같은 도면의 여러 리비전이 있으면 가장 높은 R번호를 최신 후보로 표시하고, 이전 리비전도 알려줘. 최신 후보에 PDF나 DWG 중 한쪽만 있거나 파일명 규칙에 맞지 않는 경우도 표시해줘.

파일을 삭제하거나 이름을 바꾸거나 이동하지는 마. 결과는 CSV로 저장할 수 있게 해줘. 파일정보는 외부 서버로 보내지 말고 내 컴퓨터의 브라우저 안에서만 처리해줘.

아직 구현하지 말고, 먼저 필요한 기능과 화면 구성, 파일명 규칙과 테스트 방법부터 설명해줘.

실습 A / B는 무엇을 비교할까요?

A · 파일 정리 자동화

판단 근거

확장자 + 파일명 단어

목적

파일 종류별 정리

불확실할 때

99_분류확인 → 사람 판단

B · 도면 제출 전 빠른 점검

판단 근거

도면 파일명 구조

목적

도면번호·리비전·파일형식 상태 확인

불확실할 때

확인 필요 → 사람 판단

두 프로그램 모두 파일 내부 내용을 AI가 분석하여 판단하는 프로그램이 아닙니다.

첫 요청의 목적은 코드 완성이 아니라 기능·화면·규칙·확인 방법을 제안받는 것입니다. AI 답변을 읽고 다음 단계에서 사람이 범위를 정합니다.

09 AI가 제안한 구조 검토

AI 답변과 사람이 정한 조건을 비교한 뒤 부족한 부분을 보완합니다.

AI가 제안한 내용과 자신이 요청한 범위를 비교합니다. 유지할 내용, 고칠 내용, 아직 결정하지 못한 질문을 구분합니다.

A · 파일 정리 자동화

1. 필요한 기능이 빠지지 않았나요?
2. 허용한 폴더 밖에 접근하나요?
3. 원본을 삭제하거나 보관 없이 옮기나요?
4. 기존 파일을 덮어쓰나요?
5. 애매한 파일을 강제로 분류하나요?
6. 실행·종료 방법이 있나요?
7. 오류가 나면 멈추고 원본을 유지할 수 있나요?
8. 기대 결과와 비교할 Test 기준이 있나요?

B · 도면 제출 전 빠른 점검

1. 파일명과 기본 파일정보만 읽고 도면 내부 내용은 열지 않나요?
2. 도면번호·도면명·리비전·형식을 나누는 규칙이 명확한가요?
3. R02와 R10을 숫자로 비교하고 가장 높은 값을 최신 후보로만 표시하나요?
4. 최신 후보의 PDF·DWG 짝과 같은 번호·리비전·형식 중복을 알리나요?
5. 규칙 오류, 임시파일, 숨김·지원하지 않는 파일을 구분하나요?
6. 파일 삭제·이동·이름 변경·외부 전송 없이 CSV로 결과를 남기나요?

▼ 실습 B의 역할과 파일 구성 살펴보기

파일·폴더 선택 → 파일명 분리 → 번호별 리비전 정리 → 확인할 항목 표시 → 사람 확인 → CSV 저장

```
drawing-quick-check/
├── index.html           화면 구성
├── app.js               선택 · 필터 · 화면 동작
├── rules.js             파일명 · 상태 판정 규칙
├── style.css            데스크톱 · 모바일 표시
├── sample-data.js       브라우저용 예제 목록
├── sample-manifest.json 예제 파일과 기대 결과
└── README.txt           사용 방법
```

화면 동작과 판정 규칙을 나눕니다. 파일 내부를 읽거나 파일을 바꾸는 기능은 넣지 않습니다.

사람이 확인한 내용 남기기

실습 폴더, 접근 범위, 자동 처리와 사람 판단, 오류 처리, 기대 결과를 읽습니다. 검토한 계획과 만들 파일이 일치하는지 확인한 뒤 구현을 승인합니다.

Part 3

Codex로 구현하고 직접 확인하기

이번 파트의 결과: 구현한 파일을 확인하고 프로그램을 실행해 기대 결과와 실제 결과를 비교합니다.

10 선택한 예제의 구현 명세 만들기

검토한 구조를 바탕으로 선택한 예제의 구현 범위와 확인 기준을 정합니다.

선택한 예제의 명세만 사용합니다. 작업 폴더·허용 파일·검토한 계획을 자신의 상황에 맞게 채우고, 결정되지 않은 조건을 확정된 것처럼 적지 않습니다.

A · 파일 정리 자동화

- **작업 폴더와 접근 범위**

  프로그램 폴더 안의 쓰기와, 명시적으로 선택한 자료 폴더의 최상위 파일 읽기·복사를 구분합니다.

- **자동처리 / 사람판단**

  확장자·파일명 규칙과 사람이 확인할 예외를 나눕니다.

- **원본보관과 덮어쓰기 방지**

  원본을 보관하고 같은 이름과 같은 내용의 파일을 구분합니다.

- **프로그램 구조**

  실행·종료, 새 파일 확인, 분류, 이동, 기록, 화면의 역할을 나눕니다.

- **오류 처리**

  실패한 파일의 원본을 유지하고 사용자가 이유를 확인할 수 있게 합니다.

- **Test 기준**

  실습파일 11개의 기대 결과와 수동 선택·종료 결과를 확인합니다.

▼ 실습 A 상세 구현 명세 보기

B · 도면 제출 전 빠른 점검

- **작업 폴더**
  별도 실습 폴더에 정적 HTML·JavaScript·CSS와 사용 안내를 만듭니다.
- **입력과 규칙**
  PDF·DWG·DXF의 파일명과 크기 등 기본정보만 읽습니다.
- **표시할 결과**
  최신 후보·이전 리비전·형식 확인·중복 후보·파일명 확인을 구분합니다.
- **원본 보호**
  선택한 파일을 수정하거나 서버로 업로드하지 않습니다.
- **화면과 저장**
  폴더/여러 파일 선택, 필터, 샘플, 초기화, 한글 CSV를 제공합니다.
- **기능 확인**
  파일명 시험 세트의 기대 결과와 실제 표시, CSV, 모바일 사용성을 확인합니다.

▼ 실습 B 상세 구현 명세 보기

11 Codex로 선택한 예제 구현

검토한 계획을 프로젝트 파일로 만들고 필요한 수정과 기능 확인을 이어갑니다.

자연어 첫 요청 → AI의 구조 제안 → 사람의 검토 → 구현 승인 → 수정·기능 확인 → 완성 프로그램

1. **구현 범위 확인**
   실습 폴더, 필요한 기능, 미확정 조건을 다시 읽습니다. 결정하지 않은 항목이 있으면 먼저 질문합니다.

2. **변경 예정 파일 확인**
   AI가 만들거나 수정하겠다는 파일이 검토한 구조와 맞는지 확인합니다.

3. **구현 요청**
   직전 section에서 보완한 명세를 전달하고 확인한 범위에서만 구현을 요청합니다.

4. **진행 내용 읽기**
   생성된 파일, 실행 방법, 추가 권한 요청이 계획과 맞는지 살펴봅니다.

5. **오류 원인 확인**

   오류 메시지와 실행한 동작을 함께 전달합니다. 폴더 밖 작업이나 불필요한 기능 추가로 해결하지 않도록 수정 범위를 확인합니다.

▼ **검토한 명세와 함께 전달할 구현 요청**

실행 플랫폼	OpenAI Codex / ChatGPT 또는 AI 코드 에디터
준비·실행	자연어 기획 요청 → 구조 검토 → 구현 승인 순서로 AI 코딩 도구에 입력합니다.
설계·유추 원리	승인한 명세를 폴더 구조·입출력·오류 처리·테스트로 연결하고 범위 밖 기능 추가를 금지했습니다.
AI로 작성 요청	승인한 이 명세 안에서만 구현할 프롬프트를 작성해 줘. 파일 구조·오류 처리·테스트 순서를 포함하고 불분명한 요구는 구현 전에 질문하게 해 줘.

PROMPT

구현 요청 · 선택한 예제의 명세와 함께 전달하기

```
앞에서 검토한 상세 구현 명세를 기준으로 구현해 주세요.
허용한 작업 폴더와 변경 예정 파일을 먼저 다시 보여 주세요.
명세의 빈칸이나 미확정 조건이 남아 있으면 구현 전에 질문해 주세요.
확인된 범위에서만 프로젝트 파일과 코드를 만들고, 원본 보관과 덮어쓰기 방지 조건을 지켜 주세요.
완료 후 생성·변경한 파일, 실행·종료 방법, 실제 확인한 결과와 아직 확인하지 못한 항목을 구분해 주세요.
실행하지 않은 Test를 통과했다고 쓰지 마세요.
```

수정이 필요하면 관찰한 내용을 전달합니다

"기대 결과 / 실제 결과 / 재현 순서 / 유지할 기능"을 적어 수정 요청을 합니다. 파일이 생성됐다는 답변만으로 성공을 기록하지 않고, 실행과 종료까지 확인합니다.

제공 예제는 수정과 기능 확인을 거쳤습니다

- A는 기존 분류 기능을 유지하면서 키보드 선택이 끊기지 않도록 하고, 모바일 결과 목록을 읽기 쉽게 보완했습니다.
- B는 파일명 규칙, 숫자 리비전 비교, 형식·중복 표시를 구현한 뒤 시험 파일로 화면·CSV·모바일 동작을 확인했습니다.

학습용 첫 요청은 제작 절차를 시작하는 예시입니다. 제공 프로그램 전체가 짧은 Prompt 한 번으로 만들어진 결과라는 뜻은 아닙니다. 직접 구현한 결과와 제공 예제의 실행 결과를 구분해 기록합니다.

12 완성 프로그램 실행·기능 확인

선택한 프로그램을 실행하고 직접 관찰한 결과를 기록합니다.

선택한 프로그램을 실행하고 기대 결과와 실제 결과를 비교합니다. 실행한 대상이 **직접 구현한 프로그램인지, 제공 예제인지** 먼저 기록합니다. 확인하지 못한 항목은 미확인으로 남깁니다.

A · 파일 정리 자동화 실행

Windows 교육용 로컬 프로그램입니다. Node.js는 이 프로그램의 실행환경이며 Runtime이 포함된 배포본은 아닙니다.

파일을 어떻게 넣을까요?

① 수집함에 직접 넣기

정리할 파일을 00_수집함에 넣으면 프로그램이 새 파일을 감지해 분류합니다.

② 다른 폴더에서 가져오기

기존 프로젝트 폴더를 선택하고 목록을 확인한 뒤 파일 가져오기를 누릅니다. 원본은 그대로 두고 최상위 파일을 수집함으로 복사한 뒤 같은 분류 규칙을 적용합니다.

두 방법 모두 ↓

00_수집함 → 새 파일 감지·복사 안정 확인 → 분류 규칙 확인·원본보관 → 자동분류 / 사람이 확인 → 처리 기록

가져온 원본 파일은 이동하거나 삭제하지 않습니다. 선택한 파일을 수집함으로 복사한 뒤 분류합니다. 하위 폴더는 자동으로 가져오지 않습니다.

폴더 선택은 Windows 선택창을 사용합니다. 창을 사용할 수 없으면 전체 경로를 입력하고 목록 확인을 누릅니다. 목록의 복사 대상은 분류 결과를 미리 확정한 것이 아닙니다.

1. 프로그램 ZIP과 SORT_TEST ZIP, 사용방법 README를 내려받습니다.
2. 쓰기 가능한 실습 위치에 프로그램 ZIP을 압축 해제합니다. AI_파일자동정리 폴더 전체를 유지합니다.
3. START.bat을 더블클릭합니다. 성공 안내에 나온 localhost 주소를 브라우저에서 엽니다.
4. SORT_TEST ZIP을 별도로 풉니다. 실습파일 11개만 복사해 00_수집함에 넣습니다. README.txt와 manifest.json, 하위 폴더는 넣지 않습니다.
5. 다른 자료 폴더를 사용할 때는 폴더 선택 또는 경로 입력 → 목록 확인 → 파일 가져오기를 사용합니다. 원본은 유지하고 최상위 일반 파일만 수집함으로 복사합니다.
6. 화면의 자동분류 결과와 99_분류확인을 열고 다음 표의 기대 결과와 비교합니다.

제공 예제의 주소는 `http://127.0.0.1:4174/`입니다. 사용 중이라는 안내가 나오면 다른 프로그램을 임의로 종료하지 않습니다.

SORT_TEST 기대 결과: 자동처리 9개 / 사람확인 2개. 실습파일 11개를 `00_수집함`에 넣습니다. 아래 표를 자신의 기록 문서에 옮겨 실제 결과·판정·근거를 작성합니다.

SORT_TEST 11개 · 실행 기록 양식

파일명	기대 결과	실제 결과	판정	근거
office_floorplan.png	01_CAD도면 floorplan	실행 후 기록	미확인	파일 위치 / 화면 기록
front_elevation.pdf	01_CAD도면 elevation	실행 후 기록	미확인	파일 위치 / 화면 기록
material_ref_oak.png	03_레퍼런스 material / ref	실행 후 기록	미확인	파일 위치 / 화면 기록
lighting_ref_3000K.jpg	03_레퍼런스 lighting / ref	실행 후 기록	미확인	파일 위치 / 화면 기록
render_A_v01.png	04_이미지결과 render	실행 후 기록	미확인	파일 위치 / 화면 기록
project_brief.docx	07_문서 .docx의 명확한 확장자	실행 후 기록	미확인	파일 위치 / 화면 기록
building_model.blend	05_3D모델 .blend의 명확한 확장자	실행 후 기록	미확인	파일 위치 / 화면 기록
animation_cache.abc	05_3D모델 .abc의 명확한 확장자	실행 후 기록	미확인	파일 위치 / 화면 기록
walkthrough.mp4	08_영상 .mp4의 명확한 확장자	실행 후 기록	미확인	파일 위치 / 화면 기록
image001.png	99_분류확인 용도를 알 수 없음	실행 후 기록	미확인	파일 위치 / 화면 기록
scan003.pdf	99_분류확인 용도를 알 수 없음	실행 후 기록	미확인	파일 위치 / 화면 기록

사람확인도 기대한 동작입니다

image001.png와 scan003.pdf는 처음에 99_분류확인으로 갑니다. 파일 1개의 용도를 사람이 판단해 분류를 선택하고, 실제 이동한 폴더를 확인합니다.

같은 파일을 다시 넣었을 때의 중복 안내와, 같은 이름이지만 내용이 다른 파일의 버전 증가도 확인할 수 있습니다. 자동 처리와 수동 선택 후 결과를 따로 적고 STOP.bat으로 종료합니다.

▼ 실습파일의 용도 확인

- building_model.blend와 animation_cache.abc는 분류 시험용 파일입니다. 실제 Blender 모델이나 애니메이션 캐시가 아닙니다.
- lighting_ref_3000K.jpg는 파일명 분류 시험용이며 실제 3000K 측정·보증 자료가 아닙니다.
- office_floorplan.png는 교육용 카페 평면도 복사본이며 오피스 실측도면이 아닙니다.
- scan003.pdf는 교육용 PDF 복사본, walkthrough.mp4는 1초 정지 화면 영상입니다.

▼ A의 사용 범위와 알려진 한계

- Windows 교육용 로컬 프로그램입니다. Node.js 실행환경이 필요하며 Portable Runtime 포함 버전은 아닙니다.
- 확장자와 파일명 규칙을 사용합니다. 파일 내부의 의미나 설계 사실을 분석하는 AI 분류 프로그램이 아닙니다.
- 파일 복사 완료는 잠시 상태가 변하지 않는지로 판단합니다. 큰 파일은 복사가 끝난 뒤 넣습니다.
- 같은 내용의 파일은 중복 안내 후 수집함에 남을 수 있습니다. 필요한 파일인지 확인한 뒤 사용자가 정리합니다.
- 외부 참조가 연결된 모델 전체를 복구하거나 모든 PC에서 같은 실행 결과를 보증하지 않습니다.

삭제할 때는 STOP으로 종료하고 필요한 원본·결과를 따로 보관한 뒤 프로그램 폴더를 지웁니다. OS·브라우저 캐시까지 제거한다는 뜻은 아닙니다.

B · 도면 제출 전 빠른 점검 실행

파일명과 기본정보만 브라우저 안에서 확인합니다. Node.js 설치 없이 예제를 열거나 ZIP의 `index.html`을 실행합니다.

> **이 프로그램은 무엇을 보고 점검할까요?**
>
> 이 프로그램은 도면 내부 내용을 분석하지 않습니다. 파일명에 있는 도면번호·도면명·리비전·파일형식을 사람이 정한 규칙에 따라 읽고 비교합니다.
>
> **프로그램이 표시하는 '최신 후보'는 파일명의 리비전 번호를 비교한 결과입니다. 실제 승인된 최종 도면이라는 뜻은 아니며, 최종 제출 여부는 사람이 확인합니다.**

파일명 구조 한눈에 보기

```
A-101_1층평면도_R02.pdf
```

A-101	**1층평면도**	**R02**	**PDF**
도면번호	도면명	리비전	파일형식

핵심 판단 규칙

1. 같은 도면번호끼리 묶습니다. 도면명이 같아도 번호가 다르면 다른 묶음입니다.
2. 같은 번호 안에서 유효한 R번호를 숫자로 비교합니다. R02보다 R10이 높습니다.
3. 가장 높은 R번호를 최신 후보 열에 표시합니다.
4. 낮은 R번호는 이전 리비전으로 표시합니다. 다른 확인사항이 있으면 상태는 확인 필요가 됩니다.
5. 최신 후보에 PDF와 DWG가 모두 있는지 확인합니다. DXF는 두 형식을 대신하지 않습니다.
6. 같은 도면번호·같은 숫자 리비전·같은 파일형식이 두 개 이상이면 중복 후보로 알립니다.

7. 파일명 구조를 읽지 못하면 파일명 확인으로 표시합니다. 같은 번호의 도면명이 다를 때도 확인이 필요합니다.
8. 표시한 근거를 사람이 확인합니다. 프로그램은 실제 제출 여부나 도면 내용의 적합성을 확정하지 않습니다.

대표 사례 · 실제 프로그램에서 확인한 표시

파일명	규칙에 따른 표시
리비전과 형식 함께 보기 A-101_1층평면도_R01.pdf A-101_1층평면도_R02.pdf A-101_1층평면도_R02.dwg	A-101끼리 비교 → R02 최신 후보 / R01 이전 리비전. R02에 PDF + DWG가 있어 최신 파일의 상태는 정상입니다. 제출 승인을 뜻하지 않습니다.
최신 후보가 PDF만 있을 때 A-102_2층평면도_R03.pdf	R03 최신 후보 → 상태는 확인 필요. 최신 후보 목록에 DWG가 없다는 형식 확인 안내입니다. 실제 업무의 제출 조건은 사람이 대조합니다.
파일명 구조를 읽을 수 없을 때 A101_평면최종.pdf	도면번호·도면명·리비전을 정해진 구조로 읽지 못함 → 파일명 확인. “최종”이라는 단어를 승인 근거로 사용하지 않습니다.
다른 폴더에도 같은 후보가 있을 때 원본/A-103_천장평면도_R02.pdf 복사본/A-103_천장평면도_R02.pdf 원본/A-103_천장평면도_R02.dwg	같은 번호·리비전·PDF 두 개 → 두 PDF에 중복 후보 안내 / 확인 필요. 상대 경로로 구분하며 파일 내용이나 SHA가 같다는 뜻은 아닙니다.

▼ 실제 점검 규칙 보기

지원 파일형식

PDF · DWG · DXF입니다. 확장자의 대소문자는 구분하지 않습니다. 크기·수정시각은 기본정보로 받지만 최신 후보 판단에는 사용하지 않습니다.

기본 파일명 구조

[도면번호]_[도면명]_R[숫자].[파일형식]입니다. 도면번호는 영문으로 시작하는 영문·숫자 조합 + 하이픈 + 숫자입니다. 도면명은 공백만으로 구성되면 안 되며, 이름 안의 밑줄은 허용합니다.

도면번호와 도면명

도면번호의 영문은 대문자로 맞춰 묶습니다. 도면명은 파일명 그대로 비교하므로 같은 번호에 이름이 다르면 모든 해당 파일에 확인 필요를 표시합니다.

리비전 규칙

R 뒤의 숫자를 0 이상의 정수로 읽습니다. R2와 R02는 같은 값이며 R02와 R10은 2와 10으로 비교합니다. 소수·음수·문자 또는 정확히 비교할 수 없는 큰 숫자는 파일명 확인 대상입니다.

최신 후보와 이전 리비전

선택한 유효 파일 중 같은 도면번호의 가장 높은 R번호가 최신 후보입니다. 낮은 값은 이전 리비전입니다. 선택하지 않은 파일의 존재나 번호 사이의 누락은 추정하지 않습니다.

PDF / DWG / DXF

최신 R번호에 PDF와 DWG가 모두 있는지 확인합니다. 한쪽 또는 양쪽이 없으면 최신 파일에 형식 확인 필요를 알립니다. DXF는 함께 표시하지만 PDF·DWG 짝을 대신하지 않습니다. 이전 파일에 형식 부족을 따로 표시하지 않습니다.

중복 후보

같은 번호 + 같은 숫자 R번호 + 같은 형식이 여러 개면 해당 파일 모두에 알립니다. 도면명·하위 폴더·파일 크기와 무관하며, 파일 내용을 읽거나 SHA를 비교하지 않습니다.

파일명 확인과 상태

지원 형식이어도 도면번호 구조 오류, 도면명 누락·공백, 리비전 누락·규칙 오류가 있으면 파일명 확인입니다. 나머지 상태는 확인 필요 → 이전 리비전 → 정상 순으로 표시됩니다. 최신 후보는 별도 열이며, 정상도 제출 승인이라는 뜻은 아닙니다.

제외되는 파일

PDF·DWG·DXF 외의 형식, 이름이 ~ 또는 ~$로 시작하거나 .tmp / .part / .crdownload / .bak / ~로 끝나는 파일, 이름 또는 폴더명이 점(.)으로 시작하는 경로는 제외합니다. Windows의 숨김 속성 자체를 검사하는 것은 아닙니다.

CSV에 남기는 내용

전체 점검 결과와 제외 목록을 저장합니다. 상태·파일명·상대경로·도면번호·도면명·이 파일 리비전·최신 후보·파일형식·최신 후보 보유 형식·확인사항·크기(byte)의 11개 열입니다. 한글 UTF-8 형식이며, 화면 필터로 숨긴 파일도 CSV에 포함됩니다.

AI로 프로그램을 만들었지만, 완성된 프로그램 자체가 AI로 도면을 판단하는 것은 아닙니다.

Codex / AI는 프로그램 구조 제안·코드 구현·수정·테스트를 돕습니다.

완성된 프로그램이 하는 일

완성된 프로그램은 파일명 규칙을 반복 실행합니다. AI 모델 호출, PDF·DWG 내부 분석, 이미지 인식, 도면 의미 판단을 하지 않습니다.

사람이 최종 확인할 일

사람은 원본과 제출 조건을 대조해 실제 승인본인지, 폐기된 리비전인지, 발주처 제출 대상인지, 설계자가 확정한 파일인지 확인합니다.

1. 브라우저에서 예제를 열거나 프로그램 ZIP을 풀고 index.html을 엽니다.
2. 샘플 불러오기로 표시 방식을 먼저 살펴봅니다. 샘플은 교육용 가상 파일명입니다.
3. 테스트 ZIP을 별도로 풀고 도면 폴더 선택 또는 파일 여러 개 선택을 사용합니다.
4. 전체·정상·확인 필요·이전 리비전 필터에서 파일명과 표시 이유를 읽습니다.
5. 결과 CSV를 내려받고 자신의 실제 결과와 확인사항을 기록합니다.
6. 초기화로 화면 목록을 비우고 탭을 닫습니다. 선택한 원본 파일은 바뀌지 않습니다.

시험 세트는 29개 파일명 점검용 stub입니다. 실제 CAD 도면이 아닙니다. 샘플 기대값은 선택 29개 중 점검 24개·제외 5개, 도면번호 9개, 확인 필요 파일 8개, 이전 리비전 파일 4개입니다. 상태가 겹칠 수 있어 숫자를 합쳐 전체 수로 해석하지 않습니다.

B · 주요 기대 결과와 내 실행 결과 비교

확인 사례	기대 결과	내 기록
A-101_1층평면도_R02.pdf	A-101 / 1층평면도 / R02 / PDF로 나뉨	실제 표시 / 통과·실패·미확인 / 근거
같은 번호에 R01·R02·R03	R03 최신 후보, 낮은 숫자는 이전 리비전	실제 표시 / 통과·실패·미확인 / 근거
R02와 R10	문자 순서가 아닌 숫자로 비교하여 R10 최신 후보	실제 표시 / 통과·실패·미확인 / 근거
최신 후보에 PDF와 DWG	두 형식을 보유한 것으로 표시	실제 표시 / 통과·실패·미확인 / 근거
최신 후보에 PDF 또는 DWG 한쪽만	형식 확인 필요	실제 표시 / 통과·실패·미확인 / 근거
같은 번호·리비전·형식이 다른 하위 폴더에도 존재	중복 후보로 표시하고 상대 경로로 구분	실제 표시 / 통과·실패·미확인 / 근거
번호 형식 오류·도면명 또는 리비전 누락	파일명 확인	실제 표시 / 통과·실패·미확인 / 근거
지원하지 않는 형식·임시파일·숨김 파일	제외 이유를 표시하고 도면 후보에서 제외	실제 표시 / 통과·실패·미확인 / 근거

▼ B의 사용 범위와 알려진 한계

- 파일명과 기본 파일정보만 확인합니다. 도면 내용·설계 오류·법규 적합성은 판단하지 않습니다.
- 가장 높은 R번호는 최신 후보입니다. 제출본 확정이나 실제 최신 여부를 보증하지 않습니다.
- 이 실습의 기본 규칙은 [도면번호]_[도면명]_[리비전].[확장자]이며 PDF·DWG 짝을 확인합니다. 회사마다 제출 규칙은 다를 수 있습니다.
- DXF는 함께 표시하지만 PDF·DWG 짝을 대신하지 않습니다. 번호가 건너뛰어도 누락이라고 확정하지 않습니다.
- 시험 세트는 파일명 점검용 stub입니다. 실제 설계 도면으로 사용하지 않습니다.
- 폴더 선택을 지원하지 않는 브라우저에서는 여러 파일 선택을 사용합니다. 화면 목록은 탭을 닫기 전에 CSV로 저장합니다.

13

다른 예제 빠르게 체험하기

다른 예제를 빠르게 실행하며 자동 처리와 사람 판단의 차이를 살펴봅니다.

아직 실행하지 않은 다른 예제를 열어 같은 작업 방식이 어떻게 적용되는지 살펴봅니다. 두 프로그램을 모두 처음부터 구현할 필요는 없습니다.

A를 진행했다면 → B 빠르게 체험

1. 브라우저에서 예제를 열고 샘플을 불러옵니다.
2. 최신 후보와 이전 리비전을 비교합니다.
3. 확인 필요 항목 한 개를 읽고 사람이 결정할 내용을 말해 봅니다.
4. CSV를 저장하고 원본 파일이 바뀌지 않는지 확인합니다.

B를 선택했다면 → A의 처리 흐름 확인

1. Node.js와 README를 확인하고 START.bat을 실행합니다.
2. 시험 파일을 수집함에 넣어 자동분류와 사람확인을 비교합니다.
3. 처리 결과와 원본 보관 위치를 확인합니다.
4. STOP.bat으로 종료합니다.

A는 규칙에 따라 파일을 보관·이동하고, B는 파일을 바꾸지 않고 확인할 항목을 보여줍니다. 두 예제 모두 사람이 규칙과 판단 범위를 먼저 정합니다.

14

결과 정리와 다음 차시 연결

요구사항과 판단 범위, 직접 확인한 결과를 함께 정리합니다.

1. AI와 함께 프로그램을 만들 때도 사람이 먼저 요구사항과 범위를 정합니다.
2. 확실한 반복 작업은 자동화할 수 있습니다.
3. 애매한 판단은 사람에게 남길 수 있습니다.
4. 구현한 결과는 직접 실행하고 기대 결과와 비교합니다.

이번 차시 결과 모으기

선택한 예제, 검토한 구조와 구현 명세, 생성된 파일, 직접 실행한 결과, 아직 확인하지 못한 항목을 정리합니다. 직접 구현 / 제공 예제 중 무엇을 사용했는지 함께 남깁니다.

▼ 내 실행 기록을 정리하는 Prompt

다음 차시 · 초기 디자인 방향을 빠르게 만들기

다음 차시부터는 Rapid Concept 실습으로 넘어갑니다. 재질·조명·분위기와 참고 이미지를 활용한 디자인 방향 설정을 다룹니다.